with this syntax, there are many good tutorials online such as https://javascript.info/async-await.)

In a nutshell, this syntax allows you to treat functions using `fetch` and other asynchronous APIs as if they were normal synchronous functions. Behind the scenes the JavaScript compiler will add the callbacks you'd otherwise need to write manually, and it simplifies this code quite a lot. The `loadMessages` function will be called from the `onclick` handler attached to the HTML link. It extracts the capability URI from the `href` attribute of the link and calls the `getCap` function you implemented earlier to load the list of messages in the space. It then loops through each message URI and calls the `loadMessage` function, which will again call `getCap` to load the actual message before finally creating a new row in the table for the message content. Open the capability.js file in your editor and add the two new functions from the listing.

Listing 9.11 Loading messages

```
function loadMessages(link) {
getCap(link.href, async messages => { #A
for (let messageUrl of messages) { #B
await loadMessage(messageUrl); #B
}
});
return false;
}
function loadMessage(capUrl) {
return getCap(capUrl, message => { #C
let table = document.getElementById('messages'); #C
let row = table.appendChild(document.createElement('tr')); #C
row.appendChild(document.createElement('td')) #C
.textContent = message.author; #C
```